CAPSTONE PROJECT · TECHNICAL ALIGNMENT ACTIVITY

Code Escape — Capstone Technical Alignment Report

Requirements, architecture, data and implementation evidence for the Code Escape Unity PC game, based on inspection of the actual project files.

Student	*[Student name — to be completed]*
Capstone group	*[Group name / members — to be completed]*
Course / section	*[Course code and section — to be completed]*
Instructor	*[Instructor name — to be completed]*
Institution	*[Institution — to be completed]*
Project inspected	Code Escape · Unity 6000.5.10f1 · 2D/2.5D PC game
Report prepared	11 October 2026

Evidence note. Implementation status in this report reflects the evidence inspected (project files, scripts, scenes, prefabs and package configuration) at the time of report preparation. Anything not evidenced in those files is marked Planned or listed as a gap.

Contents and How to Read This Report

Part I — Core Requirements Alignment	8 pts
Part II — Requirements-to-Architecture Mapping	8 pts
Part III — System Architecture (diagram and explanation)	12 pts
Part IV — Data and Storage Alignment	5 pts
Part V — Current Implementation Evidence	5 pts
Part VI — Next Technical Milestone	4 pts
Appendix A — Evidence still needed from the group	—

Status definitions used throughout

Status	Meaning in this report
Implemented	Working code, data or configuration for the item exists in the project files. This inspection read the files; it did **not** run the game, so runtime behaviour is "present in code, not verified at runtime" unless stated.
In Progress	Part of the item is implemented in the project files and part is still missing.
Planned	No implementation evidence in the project. The item is only intended or recommended.

Project at a glance (from the files)

Game	Code Escape: a story-driven PC game. The player (Alex) moves through six locations, and each locked obstacle or hazard opens a beginner programming question.
Languages taught	Python, PHP and SQL (`ProgrammingLanguage.cs`). One language is chosen per playthrough.
Playable levels	6: `home` , `neighborhood` , `warehouse` , `facility` , `station` , `final_area` (list in `Assets/Prefabs/Resources/GameSystems.prefab`).
Question bank	48 `QuestionData` assets: 10 obstacle questions each for Python, PHP and SQL, plus 18 hazard-challenge questions (`Assets/ScriptableObjects/Questions`).
Scenes in Build Settings	12 enabled (`00_MainMenu` … `10_Results` , `ProjectSettings/EditorBuildSettings.asset`).
Source control	The project folder is not a Git repository, so no commit history can be cited. See Appendix A.

Part I — Core Requirements Alignment

8 points

> **Rubric focus:** three to five core requirements that directly affect the technical implementation, written as system capabilities, with who needs each and why it matters.

Five requirements were selected because each one is visible in the project files and each forces a technical decision (components, data or storage).

ID	Core Requirement / Capability	Who Needs It?	Why It Matters
R1	**Present and evaluate programming questions.** The system selects a question for the chosen language, shows it, checks the player's answer against a stored solution, offers a hint, and gives feedback. It does not compile or run player code.	Student (learner); instructor (learning goal)	This is the educational core of the game. Without answer checking, obstacles cannot unlock and the game would not teach Python, PHP or SQL.
R2	**Control the player character and gate movement with obstacles.** The system reads keyboard input, moves Alex, and blocks or catches the player at programming obstacles and hazards until the related challenge is solved.	Student	It connects play to learning. Obstacles are the trigger for every question, so movement and challenge systems must exchange events reliably.
R3	**Manage level progression.** The system keeps a stable list of levels, unlocks the next level only after the required challenges are solved, shows a level map per language, and allows replay of cleared levels.	Student; capstone group (content design)	It enforces the learning order and tells the student where they are in the story. Stable level IDs are needed so saves stay valid when content changes.
R4	**Calculate and display scores and results.** The system awards points for correct answers, deducts a penalty for hints, and presents score, hints, clues, time and a rank on a results screen.	Student; instructor (performance feedback)	Scores give immediate feedback and a measurable outcome. They are the main data any later report or leaderboard would use.
R5	**Persist and restore progress.** The system saves progress automatically and restores language, level, score, solved challenges and settings when the game is reopened. Linking progress to a student identity is a possible extension (see note).	Student; capstone group (testing and demonstration)	Play sessions are interrupted. Without persistence, students would restart the story each time and the group could not demonstrate continuing progress.

Note on student accounts. The instructor prompt mentions student accounts "where applicable". The project files contain no login, registration, user or authentication code (Part V, E6), and no document in the project states that accounts are a requirement. Accounts are therefore **not** counted among R1–R5. They appear as a Planned extension of R5 and are examined again in Part VI. The group should confirm whether accounts are an actual capstone requirement (Appendix A).

Part II — Requirements-to-Architecture Mapping

8 points

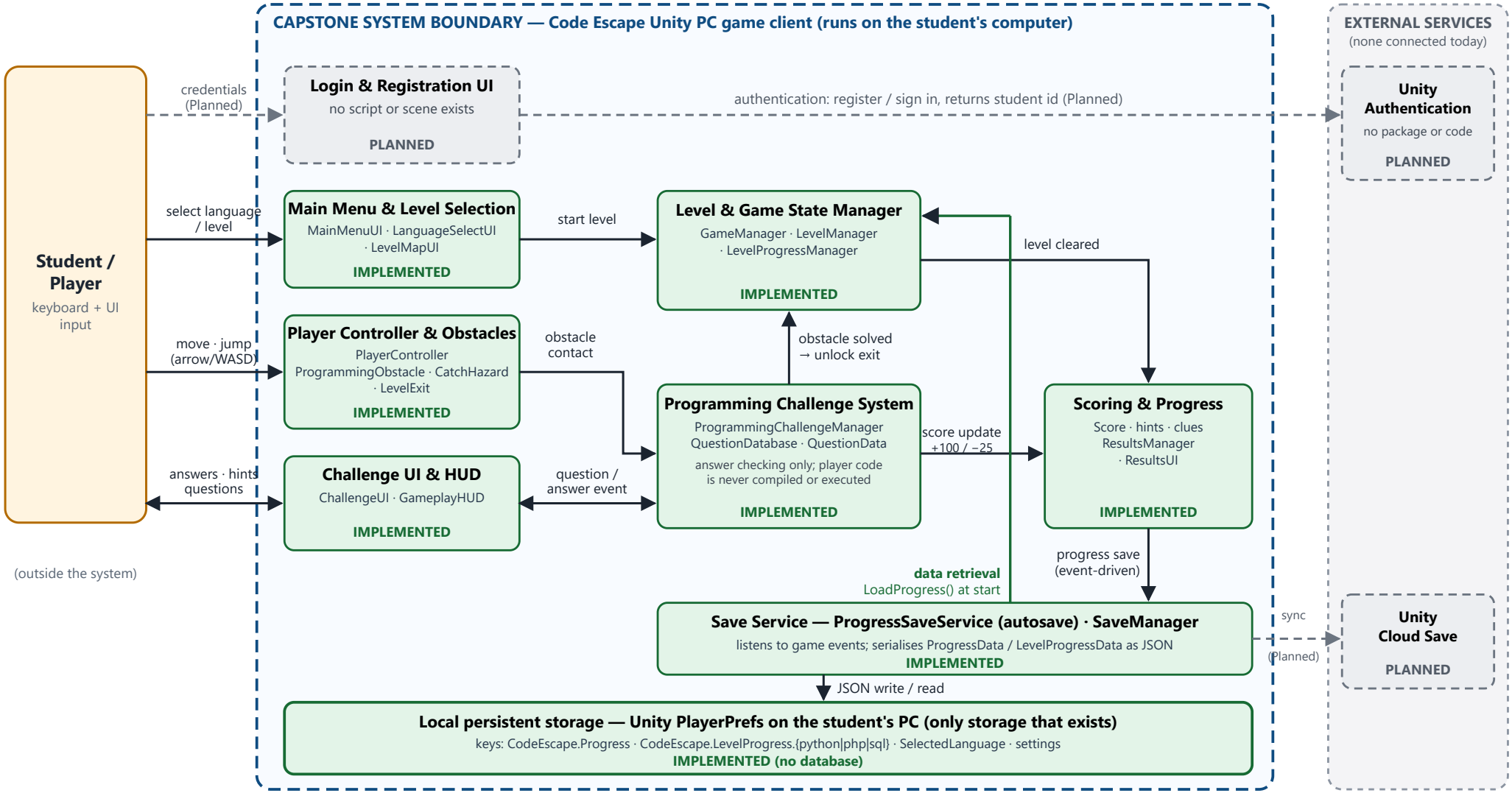
> **Rubric focus:** every Part I requirement mapped to specific responsible components, the data needed, and an evidence-based status.

Component names are the actual C# classes and assets under `Assets/Scripts` and `Assets/ScriptableObjects`.

Requirement	Responsible Component/s	Data Needed	Current Status
R1 Present and evaluate programming questions	`ProgrammingChallengeManager` (opens challenge, `SubmitChoice` / `SubmitTypedAnswer`, hint); `QuestionData` (`IsCorrectChoice`, `IsCorrectTypedAnswer`); `QuestionDatabase`; `ChallengeUI`; `LanguageManager`	Question id, language, topic, difficulty, type, questionText, code, choices, correctChoiceIndex, acceptedAnswers, caseSensitive, hint, explanation; submitted answer; correct/incorrect result	**Implemented** Code and 48 question assets present; not run.
R2 Control player and gate movement with obstacles	`PlayerController` (side-view and top-down modes); `CameraFollow2D` / `CameraController`; `ProgrammingObstacle`; `CatchHazard`; `HazardMotion`; `LevelExit`; `GameManager` + `GameState`	Movement input, movement mode, grounded state; obstacleId, required question, hazard type and difficulty; GameState (Playing, Challenge, Paused, Cutscene …); obstacle solved flag	**Implemented** Nine obstacles are placed in scenes 03–08. Hazard scripts are the most recently edited files.
R3 Manage level progression	`LevelManager` (`LevelInfo` list, unlock/complete); `LevelProgressManager`; `LevelMapUI` + `LevelMapNode`; `LevelExit`; `SceneTransitionManager`; `SceneNames`	levelId, sceneName, displayName, requiredChallenges, unlockedAtStart; unlockedLevels, completedLevels, activeLevel, lastPlayedLevel, per-level checkpoints	**Implemented** Level map (`01b_LevelMap`) and progress logic present; not run.
R4 Calculate and display scores and results	`ProgrammingChallengeManager` (`Score`, `HintsUsed`, `ScoreChanged`); `GameplayHUD` + `ScoreLabel`; `ResultsManager`; `ResultsUI`; `ClueManager`	score (+100 per correct answer, −25 per first hint on a question), hintsUsed, completed challenges / total, clues collected / total, timePlayed, rank, completion status	**Implemented** Scoring and results screen exist. Score is a single running total; no per-level score.
R5 Persist and restore progress	**Existing:** `SaveManager` (PlayerPrefs + JSON); `ProgressSaveService` (autosave); `ProgressData`; `LevelProgressData`; `SettingsData`; `SettingsManager`; `MainMenuUI` (Continue). **Not present:** Login/Registration UI, Unity Authentication, Unity Cloud Save	Existing: language, storyStage, currentLevel, activeLevel, completed/unlocked levels, completedChallenges, completedQuestions, hintedQuestions, collectedClues, score, hintsUsed, timePlayed, settings. Not present: student id, credentials, cloud copy	**In Progress** Local save **Implemented**; student identity and cloud storage **Planned**.

Part III — System Architecture

12 points



Legend: solid green box = Implemented (present in project files, not run during this inspection) · dashed grey box / dashed arrow = Planned (no implementation evidence) · solid arrows = communication that exists in code (C# events and singleton references) · green arrow = data retrieval. No component was found that is only partly built, so none is drawn In Progress.

Architecture explanation (Part III)

The student interacts with the Unity PC client through keyboard and UI input: menu and level-map selection (`MainMenuUI`, `LevelMapUI`), movement (`PlayerController`) and answering questions in `ChallengeUI`. Main processing happens on the client, inside a persistent `GameSystems` object. `ProgrammingChallengeManager` compares answers with stored solutions, without executing player code, and updates the score, while `LevelManager` and `LevelProgressManager` unlock levels. Persistent data is stored on the student's own device in PlayerPrefs as JSON through `SaveManager`. No database, authentication or cloud service exists in the project. Components communicate through C# static events (such as `ChallengeCompleted`, `ScoreChanged` and `LevelUnlocked`) and singleton `Instance` references; `ProgressSaveService` listens to these events to autosave. File modification dates show that hazard-triggered challenges and the level map were edited most recently, so they appear to be the area under active work (the group should confirm). The next component to implement is student identity, meaning a Login/Registration UI backed by a storage interface, which Unity Authentication and Cloud Save would later depend on.

Evidence behind each point

- **Interaction:** `PlayerController.cs` uses the Input System package; UI scripts under `Assets/Scripts/UI`.
- **Processing and communication:** events declared in `ProgrammingChallengeManager.cs`, `LevelManager.cs`, `ClueManager.cs`, `StoryManager.cs`; subscribed in `ProgressSaveService.cs` (`OnEnable`).
- **Storage:** `SaveManager.cs` calls only `PlayerPrefs`. A search of `Assets/Scripts` found no use of Unity Services, web requests, SQLite, Firebase, file I/O, login or registration.
- **Current / next work:** inference from file timestamps (10 Oct 2026) and from the absence of any identity code. It is not a statement from the group.

Part IV — Data and Storage Alignment

5 points

> **Rubric focus:** relevant entities and fields, why each must persist, and where it is stored, separating existing storage, planned storage, temporary session data and persistent data.

> **Honest summary:** the project has **no database**. The only implemented storage is Unity `PlayerPrefs` on the local computer (on Windows standalone builds Unity keeps PlayerPrefs in the current user's registry). All other storage below is Planned.

Gaps found in the current storage design. (1) Every user of one computer account shares the same save slot. (2) Finishing the story calls `ClearSave()`, so the final score and results are not kept after the Results screen. (3) Score is one running total with no per-level or per-question score history.

Data / Entity	Important Fields	Why It Must Persist	Storage
Selected language Persistent	`language` (Python / PHP / SQL)	Decides which question set and which level-map progress belong to the player on the next launch.	Existing: PlayerPrefs key `CodeEscape.SelectedLanguage`
Playthrough progress (`ProgressData`) Persistent	version, language, storyStage, currentLevel, currentObjective, activeLevel, completedEvents, unlockedLevels, completedLevels, completedChallenges, completedQuestions, hintedQuestions, collectedClues, **score**, hintsUsed, timePlayed	Lets the student continue and keeps solved challenges, clues and score.	Existing: one JSON string in PlayerPrefs key `CodeEscape.Progress`. Deleted when the story is finished (`ResultsManager`).
Level-map progress (`LevelProgressData`) Persistent	version, language, completedLevels (stable levelIds), lastPlayedLevel, checkpoints[levelId, ProgressData state]	Keeps cleared levels unlocked and allows replay or skip-to from the level map, separately for each language.	Existing: one JSON string per language, key `CodeEscape.LevelProgress.<language>`
Player settings (`SettingsData`) Persistent	musicVolume, sfxVolume, fullscreen, resolutionWidth, resolutionHeight	Restores the student's audio and display preferences.	Existing: separate PlayerPrefs keys (`CodeEscape.MusicVolume` etc.)
Question bank (`QuestionData`) Design-time content	id, language, topic, difficulty, questionType, questionText, code, choices, correctChoiceIndex, acceptedAnswers, hint, explanation	Content authored by the group; read-only at runtime. It is not player data and is not saved by the player.	Existing: 48 ScriptableObject assets in `Assets/ScriptableObjects/Questions`, referenced by a `QuestionDatabase`
Active challenge / game state Temporary session data	activeQuestion, activeSolved, selected answer, GameState, current hazard, dialogue/cutscene queue	Needed only while playing; it is rebuilt at each launch. Persisting it is not required.	Memory only (managers in the `GameSystems` object)
Student account and cloud progress copy Planned	studentId, username (credentials handled by the authentication service, not by game code); cloud copy of the playthrough and level-map progress keyed by studentId	Would tie progress to a person instead of a device and allow resuming on another computer. Needed only if the group confirms accounts as a requirement.	Planned: Unity Authentication and Unity Cloud Save. Nothing exists today.

Part V — Current Implementation Evidence

5 points

> **Rubric focus:** at least three pieces of genuine evidence with file names, current status and the requirement or component supported.

Evidence was gathered by reading project files. The game was **not run** for this report, so "Implemented" means "present in code or configuration, not verified at runtime".

Evidence	Current Status	Requirement / Component Supported
E1 Local save system. `Assets/Scripts/SaveSystem/SaveManager.cs` stores language, settings, progress and per-language level progress in PlayerPrefs; progress is serialised with `JsonUtility`. `ProgressSaveService.cs` saves after clue, challenge, score, story, unlock and scene events, every 30 s during play, on pause and on quit. `ProgressData.cs` and `LevelProgressData.cs` define the saved fields.	**Implemented** in code; save/load round trip not verified	R5; components SaveManager, ProgressSaveService, ProgressData, LevelProgressData
E2 Question evaluation and scoring. `Programming/ProgrammingChallengeManager.cs` defines `PointsPerCorrectAnswer = 100` and `HintPenalty = 25`, and routes answers to `QuestionData.IsCorrectChoice` / `IsCorrectTypedAnswer`. The code comment in `QuestionData.cs` states player input is only compared with the stored answer and never executed. 48 question assets exist: 16 per language (10 obstacle + 6 hazard questions).	**Implemented** in code and data assets; not run	R1 and R4; ProgrammingChallengeManager, QuestionData, QuestionDatabase, ChallengeUI
E3 Player and obstacles. `Player/PlayerController.cs` (Rigidbody2D movement, side-view and top-down). `Obstacles/ProgrammingObstacle.cs`, `CatchHazard.cs`, `HazardMotion.cs`. Scenes `03_Home` to `08_FinalArea` contain nine obstacle IDs: home_hallway_door, neighborhood_gate, warehouse_security, facility_door_a/b, station_lock, final_lock_a/b, final_door.	**Implemented** in code and scenes; not run	R2; PlayerController, ProgrammingObstacle, CatchHazard, LevelExit
E4 Level progression and level map. `Prefabs/Resources/GameSystems.prefab` lists six levels with stable `levelId`, `sceneName` and `requiredChallenges` (1+1+1+2+1+3 = 9). `Game/LevelManager.cs`, `LevelProgressManager.cs`, `UI/LevelMapUI.cs`, scene `01b_LevelMap.unity`. `EditorBuildSettings.asset` enables all 12 scenes.	**Implemented** in code, prefab and scene; not run	R3; LevelManager, LevelProgressManager, LevelMapUI, SceneTransitionManager
E5 Results screen. `Game/ResultsManager.cs` collects language, score, challenges, clues, hints, time and rank; `UI/ResultsUI.cs` displays them in scene `10_Results.unity`.	**Implemented** in code and scene; not run	R4; ResultsManager, ResultsUI

Evidence	Current Status	Requirement / Component Supported
E6 Absence of accounts, authentication, cloud and database. `Packages/manifest.json` lists no `com.unity.services.*` package (no Authentication, no Cloud Save). A text search of `Assets/Scripts` for Unity.Services, CloudSave, UnityWebRequest, Firebase, Sqlite, login, register and password returned no matches except PlayerPrefs. No login or registration scene exists among the 12 scenes.	**Planned** (nothing implemented)	R5 extension; Login/Registration UI, Unity Authentication, Unity Cloud Save
E7 Automated tests. The Unity Test Framework package is installed (`manifest.json`), but no test scripts or assembly definitions were found under `Assets` .	**Not found**	Verification of R1–R5. No automated test result can be cited.

Screenshots

No screenshots, exported runtime logs or test results were available in the project folder, and none are invented here. The frames below are placeholders for the student to fill (details in Appendix A).

[Screenshot S1 — Main Menu with Continue / Play Adventure — to be supplied]

[Screenshot S2 — Level Map showing locked / available / completed levels — to be supplied]

[Screenshot S3 — Programming challenge open at an obstacle (ChallengeUI) — to be supplied]

[Screenshot S4 — Results screen, and Unity Console log showing a save/load — to be supplied]

Part VI — Next Technical Milestone

4 points

> **Rubric focus:** one milestone chosen from the real gap, with the gap, the milestone, the reason and the expected output.

Current Gap	Progress is saved only on the local computer and is not tied to any student. `SaveManager` is a static class that calls PlayerPrefs directly, so it cannot be replaced by a different storage without editing every caller. There is no Login/Registration UI, no student identifier, and no service package in `manifest.json`. In addition, the save and load path has not been verified at runtime, and finishing the story deletes the saved result.
Next Technical Milestone	**Student identity and a swappable progress store, verified locally.** • Add a small storage interface (for example `IProgressStore`) with the current PlayerPrefs code as its first implementation. • Introduce a `studentId` and key every saved record by it, with a minimal Login/Registration scene that produces the ID (a local ID first). • Run and record a save, quit, relaunch, continue test for R3 and R5.
Reason	Unity Authentication and Cloud Save both depend on having a student ID and on a storage layer that can change backend. Adding either service before those two pieces would mean rewriting the save code twice. The milestone adds no new technology, it uses the existing, already coded save system, and it closes the verification gap shown in E1 and E7. It also tests whether the group really needs accounts before spending effort on cloud services.
Expected Output	• Source: the interface, the PlayerPrefs implementation, and a Login/Registration scene with its script. • Save data keyed by `studentId`; two different IDs on one computer keep separate progress. • Evidence for the next report: screenshots of the login scene and of the Continue flow, a Console log of save and load, and a short written test table (steps and observed results). • An updated Part III diagram with the Login UI marked Implemented and Unity Authentication/Cloud Save still Planned, or Implemented only if they are added later.

Assumption to confirm. This milestone assumes the capstone requires student accounts. If the group decides accounts are out of scope, the alternative milestone is the same runtime verification of the local save plus keeping the final score instead of clearing it in `ResultsManager`.

Appendix A — Evidence Still Needed From the Group

The following could not be established from the files and must be supplied or confirmed by the student. Nothing below has been assumed in the report.

#	Missing item	Why it is needed
1	Screenshots S1–S4 of the running game (main menu, level map, challenge, results).	Part V can then show runtime evidence rather than code evidence only.
2	A Unity Console log or short notes from running a save → quit → relaunch → Continue test.	Confirms R5 and the local save at runtime.
3	Confirmation (or the course document) stating whether student accounts and cloud storage are required.	Decides whether the Part VI assumption holds.
4	Git or other source-control history, if the group uses it elsewhere. The project folder is not a repository.	Allows real commit references; none were invented.
5	Whether the project is linked to a Unity Cloud project (Project Settings → Services) and whether any Unity Dashboard setup exists.	Needed before any status for Authentication or Cloud Save could change from Planned.
6	Which component the group considers "currently in progress".	The report infers it from file dates only.
7	Student name, group members, course, instructor and institution (cover placeholders).	Not invented.

Method and limits

- Files read: all scripts under `Assets/Scripts/SaveSystem`, `Programming`, `Game` (key files), `Player` and `Obstacles`; `Packages/manifest.json`; `ProjectSettings/EditorBuildSettings.asset`; `GameSystems.prefab`; scene files (obstacle IDs); ScriptableObject question assets (counts).
- The game and the Unity Editor were not run for this report, and no project file was changed.